

Integrabilidad (Integrability)

Integrabilidad

- Capacidad de integrarse con otros sistemas (o sus propios componentes entre sí)

Factores que afectan Integrabilidad

- Interfaces
 - Puntos de comunicación entre dos componentes
- Dependencias entre dos componentes
 - Sintácticas
 - Semántica
 - Requieren conocer un mismo protocolo, formato, unidad de medida, esquema de datos, etc.
 - Recursos compartidos
 - Secuencia temporal

Tácticas para Integrabilidad

- Limitar dependencias
 - Encapsular
 - Usar intermediarios (Indirection)
 - **Adhesión a estándares**
 - Abstraer servicios comunes
- Adaptar
 - **Descubrimiento (Discover)**
 - **Adaptar interfaces**
 - **Configurar comportamiento**
- Coordinar
 - **Orquestar**
 - **Administrar recursos**

Adhesión a estándares

- Principalmente estándares de comunicación
 - Ejemplo
 - TCP/IP
 - HTTP
 - HL7-FHIR (Intercambio de datos de salud)
 - ISO 20022 (Intercambio de datos financieros)
 - UN/EDIFACT (Intercambio documentos electrónicos entre organizaciones)

Descubrimiento (Discover)

- Crear "índices" para ubicar recursos desplegados en el sistema
- Ejemplos
 - DNS

Adaptar interfaces

- Modificar interfaces para facilitar comunicación entre componentes o limitar su uso a lo mínimo necesario
- Ejemplos
 - Interception Filters en servicios web
 - Traducción de formatos. Ej: XML -> JSON
 - API Gateways

Configurar comportamiento

- Pariente cercano de "Defer Binding"
- Parametrizar un componente para comportarse de manera diferente en ejecución, según se necesite
- Ejemplos
 - Archivos de configuración
 - Flags de compilación
 - Make + configure

Orquestar

- Coordinar ejecución de múltiples servicios, de forma que no necesiten conocerse entre sí
- Ejemplos
 - Administradores de pipelines CI/CD (ej. Github Actions)
 - Orquestación de Kubernetes o Docker compose
 - Sistemas BPM

Administrar recursos

- Mediar entre los procesos y los recursos computacionales
- Ejemplos
 - Sistemas operativos
 - Thread pools (Java, .NET)
 - Garbage collectors
 - ARINC 653 (Safety-Critical Avionics)
 - Particionamiento estricto de recursos (memoria, cpu, etc.) para componentes de Aviónica

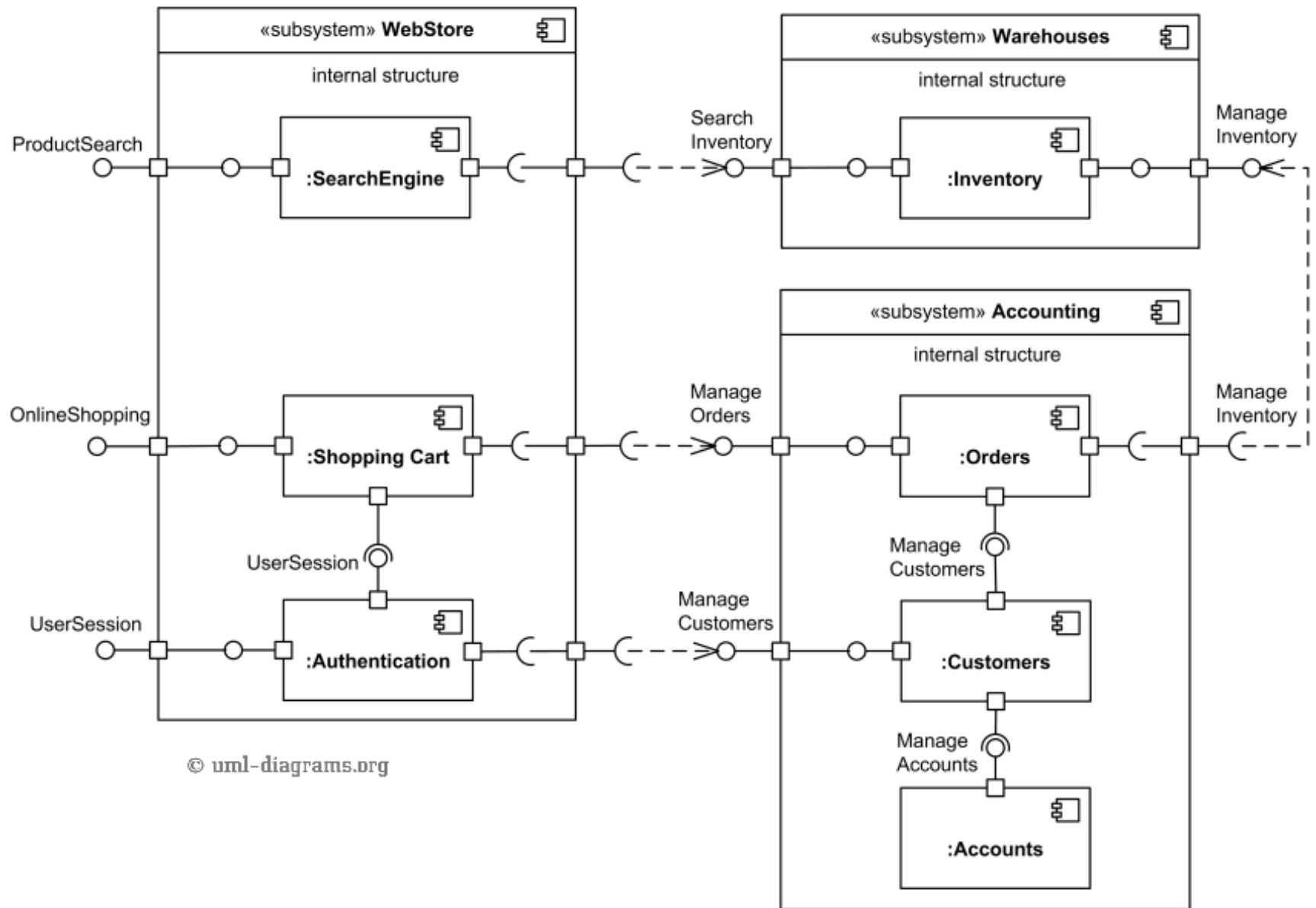
Patrones de Integrabilidad

Wrapper / Adapter / Bridge / Mediator

- Componente A que hace de intermediario aotro B
- Solo A debería acceder a B
 - Los demás componentes deberían usar A para acceder a B
- Tácticas
 - Adaptar interfaz

Service-Oriented Architecture (SOA)

- Forma más avanzada del patrón Service-Based
- Múltiples servicios, desplegados independientemente, con **roles**:
 - Proveedores
 - Proveen una interfaz que puede ser consumida
 - Consumidores
 - Indican sus "necesidades" de interfaz que son satisfechas por un proveedor



Hexagonal Architecture (HA)

- Componentes
 - Núcleo: contiene la lógica del negocio
 - Puertos: Describe interfaces que provee y que requiere
 - Adaptadores: implementación del protocolo subyacente a los puertos (Ej. REST, SOAP, etc.)
- HA puede usarse para implementar cada servicio individual

SOA

- Ventajas
 - Estándarización de interfaces
 - Reutilización
 - Independencia entre servicios
- Desventajas
 - Mayor complejidad en protocolos de comunicación

Descrubrimiento dinámico (Dynamic discovery)

- Tácticas: Discovery, con un énfasis en tiempo de ejecución
- Ejemplos:
 - Netflix Eureka
 - Apache ZooKeeper
 - Kubernetes CoreDNS
- Ventajas
 - Flexibilidad
- Desventajas
 - Complejidad adicional en registro y de-registro de recursos